

Car Price Predictor

Classification Model - Full Documentation

Random Forest Classifier

Accuracy - Precision - Recall - F1 Score

Generated: July 12, 2026

1. What Changed: Regression to Classification

The original version of this app predicted an exact car price in GHS (e.g., GHS 2,350,000). This is called regression -- predicting a continuous number. The new version predicts which price bracket (category) the car falls into. This is called classification -- predicting a label.

Why the change? Classification lets us use metrics like accuracy, precision, and recall, which are easier to explain to a non-technical audience than MAE, RMSE, and R-squared.

The Four Price Categories:

- Budget: Cars priced below GHS 300,000
- Mid-Range: Cars priced between GHS 300,000 and GHS 800,000
- Premium: Cars priced between GHS 800,000 and GHS 1,800,000
- Luxury: Cars priced above GHS 1,800,000

Instead of asking "What is the exact price?", the model now answers "Is this car Budget, Mid-Range, Premium, or Luxury?" This transforms the problem from "how far off is the number?" to "did we guess the right category?"

2. Type of Learning: Supervised Classification

This project uses Supervised Learning -- the model learns from labeled examples. Each car in the training data has features (brand, year, mileage, fuel type, transmission) paired with a correct label (Budget, Mid-Range, Premium, or Luxury).

Within supervised learning, this is a Classification task because the output is a discrete category label, not a continuous number.

- Supervised Learning -- Training data includes both input features and correct answers.
- Multi-Class Classification -- The output is one label chosen from four possible categories.
- Batch Learning -- The model is trained once on historical data, not updated incrementally.
- Model-Based Learning -- Patterns are stored in the model coefficients, not by memorising training examples.

3. Algorithm: Random Forest Classifier

A Random Forest is like a committee of decision trees. Each tree asks a series of yes/no questions about the car (e.g., "Is the year after 2015? "Is the mileage below 50,000?") and follows branches until it reaches a final category. One tree alone might make mistakes, but when 100 trees vote, the majority answer is usually correct.

How it works, step by step:

- Step 1: The algorithm creates 100 different "mini" datasets by randomly sampling from the original data (with replacement). Some cars appear multiple times; some are left out.
- Step 2: For each mini-dataset, a decision tree is grown. At each split point, the tree asks the question that best separates the price categories (e.g., "Is mileage less than 80,000?").
- Step 3: Each tree votes independently. When you ask "what category is this Toyota?", all 100 trees give their answer.

- Step 4: The category with the most votes wins. If 70 trees say "Mid-Range" and 30 say "Budget", the prediction is "Mid-Range".

This approach is called ensemble learning -- combining many weak learners into one strong predictor. It is like asking 100 experts instead of just one.

Why Random Forest instead of a single decision tree?

- Less overfitting -- A single tree can memorise the training data and fail on new data. Averaging 100 trees smooths out these quirks.
- Handles mixed data well -- Works with both numeric features (year, mileage) and categorical features (brand, fuel type) without needing scaling.
- Built-in feature importance -- The model tells you which features matter most (e.g., "year" might be more important than "fuel type").
- Robust to noise -- A few incorrectly labeled cars in the training data will not throw off the whole model.

4. Evaluation Metrics: The Four Key Questions

After training, we test the model on 20% of the data it has never seen -- just like a final exam with questions the student did not practice. Four metrics tell us how well it did.

4.1 Accuracy -- "How many predictions were correct?"

Accuracy is the simplest metric. Count how many cars the model classified correctly, then divide by the total number of cars.

$$\text{Accuracy} = \text{Correct Predictions} / \text{Total Predictions}$$

Example: The model sees 100 cars. It correctly labels 78 of them. Accuracy = 78%.

In plain English: "Out of every 100 cars, the model gets about 78 right."

- Strength: Easy to understand. One number tells the overall story.
- Weakness: Can be misleading if categories are imbalanced. If 80% of all cars are Mid-Range, a lazy model that always says "Mid-Range" gets 80% accuracy without learning anything useful.

Analogy: A student taking a multiple-choice test

Accuracy = number of correctly answered questions / total questions.

A score of 85% on a 100-question test means 85 correct answers.

4.2 Precision -- "When we make a claim, how often are we right?"

Precision focuses on the model's positive predictions. For each category, it asks: 'Of all the times the model said this car belongs to this category, what percentage of the time was it actually correct?'

$$\text{Precision} = \text{True Positives} / (\text{True Positives} + \text{False Positives})$$

True Positive: Model said "Luxury" and the car really IS Luxury. (Correct)

False Positive: Model said "Luxury" but the car is actually Budget. (False alarm)

- High precision means when the model claims a category, you can trust it. It rarely cries wolf.
- Low precision means the model over-claims -- it says "Luxury" too often and is frequently wrong.
- The app uses weighted precision -- each category's precision is averaged, weighted by how many

actual cars belong to that category.

Analogy: A doctor diagnosing a disease

Precision = "Of all the patients the doctor said are sick, how many actually have the disease?"

High precision: The doctor is cautious. When she says you are sick, you almost certainly are.

Low precision: The doctor scares many healthy people unnecessarily.

4.3 Recall -- "How many real cases did we find?"

Recall focuses on catching all cases. For each category, it asks: "Of all the cars that truly belong to this category, what percentage did the model correctly identify?"

$$\text{Recall} = \text{True Positives} / (\text{True Positives} + \text{False Negatives})$$

False Negative: Model said "Mid-Range" but the car is actually Luxury. (Missed it)

- High recall means the model misses very few cars. It catches most of them.
- Low recall means the model overlooks many cars in that category -- they slip through undetected.
- The app uses weighted recall, averaging across categories by their size.

Analogy: Airport security screening

Recall = "Of all passengers carrying prohibited items, what % did security catch?"

High recall: Security catches most threats (good for safety).

Low recall: Many prohibited items slip through (dangerous).

Note: High recall often means more false alarms too -- the trade-off with precision.

4.4 F1 Score -- "The balanced overall grade"

Precision and recall pull in opposite directions. A cautious model has high precision but low recall. An aggressive model has high recall but low precision. F1 Score combines them into one number that is high only when BOTH are high.

$$F1 = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

- F1 ranges from 0 (terrible) to 1 (perfect).
- F1 uses the harmonic mean, not the regular average. This punishes extremes -- if one is 90% and the other is 10%, the F1 is around 18%, not 50%.
- Think of F1 as the models "overall report card grade."

Why harmonic mean matters

Regular average of 90% and 10% = 50% (looks OK, deceiving).

Harmonic mean (F1) of 90% and 10% = 18% (exposes the weakness truthfully).

You cannot hide a bad precision behind a good recall, or vice versa.

5. The Precision-Recall Trade-off

There is always a tug-of-war between precision and recall. Improving one usually hurts the other. Which one matters more depends on what you are using the model for.

Scenario	Prioritise	Why
Spam email filter	Precision	False alarm = a real email goes to spam. Very annoying for users.
Cancer screening test	Recall	Missing a cancer is fatal. False alarms just mean more tests.

Car price categorisation	Balanced (F1)	Neither type of mistake is catastrophic. Overall correctness matters most.
Fraud detection	Recall	Missing fraud costs money. False alarms need human review but are affordable.

For our car price app, we want balanced performance across all four categories. F1 Score is the best single metric to optimise.

6. The Confusion Matrix: Seeing Every Mistake

A confusion matrix is a table showing exactly which categories the model confuses with each other. It goes beyond a single number and reveals patterns in the errors.

	Predicted YES	Predicted NO
Actual YES	True Positive (correct)	False Negative (miss)
Actual NO	False Positive (false alarm)	True Negative (correct)

For multi-class problems (4 categories), the matrix is 4x4 instead of 2x2. Each cell shows how many cars of actual category X were predicted as category Y.

Reading a confusion matrix

Row = actual category (the truth)

Column = predicted category (what the model said)

Diagonal cells (top-left to bottom-right) = correct predictions.

Off-diagonal cells = mistakes. The cell tells you which two categories the model confused.

The confusion matrix answers: "When the model makes a mistake, which categories does it confuse the most? Is it mixing up Budget and Mid-Range, or confusing Premium with Luxury?"

7. Code Walkthrough (Simplified)

7.1 Converting Price to Categories

```
PRICE_BINS = [0, 300000, 800000, 1800000, 9999999]
PRICE_LABELS = ["Budget", "Mid-Range", "Premium", "Luxury"]

df["price_category"] = pd.cut(df["price"],
                              bins=PRICE_BINS, labels=PRICE_LABELS)
```

The continuous price column is chopped into four bins using `pd.cut()`. Each car gets a label like "Premium" based on which bin its price falls into. This new column becomes the target (y) the model learns to predict.

7.2 Training the Random Forest

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

`n_estimators=100` means 100 decision trees vote. `random_state=42` ensures the same trees are built every time the app runs (reproducible). The `.fit()` method teaches the model by showing it features (`X_train`) and the correct category for each car (`y_train`).

7.3 Calculating Metrics

```
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, f1_score
)

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred,
    average="weighted")
recall = recall_score(y_test, y_pred,
    average="weighted")
f1 = f1_score(y_test, y_pred,
    average="weighted")
```

The model makes predictions on the 20% test set (cars it never saw during training). Each metric function compares the predicted labels against the true labels. `average="weighted"` means each category contributes to the final score proportional to its size -- larger categories have more influence.

7.4 Making a Prediction with Confidence

```
predicted_category = model.predict(input_row)[0]
probabilities = model.predict_proba(input_row)[0]
confidence = dict(zip(model.classes_, probabilities))
```

`model.predict()` returns the winning category. `model.predict_proba()` returns the confidence score (0 to 1) for EACH possible category. This reveals whether the model is 90% sure about "Premium" or only 40% sure with "Mid-Range" at 35%.

8. Quick Reference: All Four Metrics

Metric	Question It Answers	Range	When Low	When High	Best For
Accuracy	How many predictions were correct overall?	0 to 1	Model is often v	Model is usually	Balanced data
Precision	When we predict a category, how often right?	0 to 1	Many false alarm	Trustworthy pre	Avoid false alarms
Recall	How many real cases did we catch?	0 to 1	We missed mar	We found most	Avoid misses
F1 Score	Balanced quality of P and R combined?	0 to 1	Either P or R is	Both P and R ai	Overall quality

9. A Worked Example

Imagine the test set has 100 cars:

- 25 Budget, 30 Mid-Range, 25 Premium, 20 Luxury (these are the true labels)

The model makes predictions and we tally the results:

Actual \ Predicted	Budget	Mid-Range	Premium	Luxury
Budget (25 cars)	20	4	1	0
Mid-Range (30 cars)	3	24	2	1
Premium (25 cars)	0	3	19	3
Luxury (20 cars)	0	1	3	16

Total correct predictions (diagonal): $20 + 24 + 19 + 16 = 79$ out of 100.

Accuracy = $79 / 100 = 79\%$

Precision for each category: Budget = $20/23 = 87\%$, Mid-Range = $24/32 = 75\%$, Premium = $19/25 = 76\%$, Luxury = $16/20 = 80\%$

Weighted Precision = $(25 \cdot 0.87 + 30 \cdot 0.75 + 25 \cdot 0.76 + 20 \cdot 0.80) / 100 = 79\%$

Recall for each category: Budget = $20/25 = 80\%$, Mid-Range = $24/30 = 80\%$, Premium = $19/25 = 76\%$, Luxury = $16/20 = 80\%$

Weighted Recall = $(25 \cdot 0.80 + 30 \cdot 0.80 + 25 \cdot 0.76 + 20 \cdot 0.80) / 100 = 79\%$

F1 Score for Budget = $2 \cdot (0.87 \cdot 0.80) / (0.87 + 0.80) = 83\%$ (then weighted average across all = 79%)

Notice: Accuracy, weighted Precision, weighted Recall, and weighted F1 all happen to be around 79% in this balanced example. When categories are imbalanced, these numbers can diverge significantly -- which is why it is important to check all four metrics, not just accuracy.

10. Summary: What We Built

The Car Price Predictor is now a multi-class classification system powered by a Random Forest classifier. It takes five car attributes (brand, year, mileage, fuel type, transmission) and predicts which of four price brackets the car belongs to.

Key takeaways:

- Random Forest = 100 decision trees voting together. Each tree looks at the car features and casts a vote for Budget, Mid-Range, Premium, or Luxury. Majority wins.
- Accuracy = overall correctness. Precision = trustworthiness of claims. Recall = ability to catch all cases. F1 = balanced single score.
- The app also shows confidence scores -- not just the winning category, but how strongly the model leans toward each option.
- All metrics are calculated on a held-out 20% test set the model never saw during training. This gives a realistic estimate of real-world performance.